

NotebookLM LINE Bot Platform — 專案交接文件

專案來源：<https://github.com/YwY170/notebooklm-line>

撰寫日期：2026-05-22

撰寫人：Wendy YU

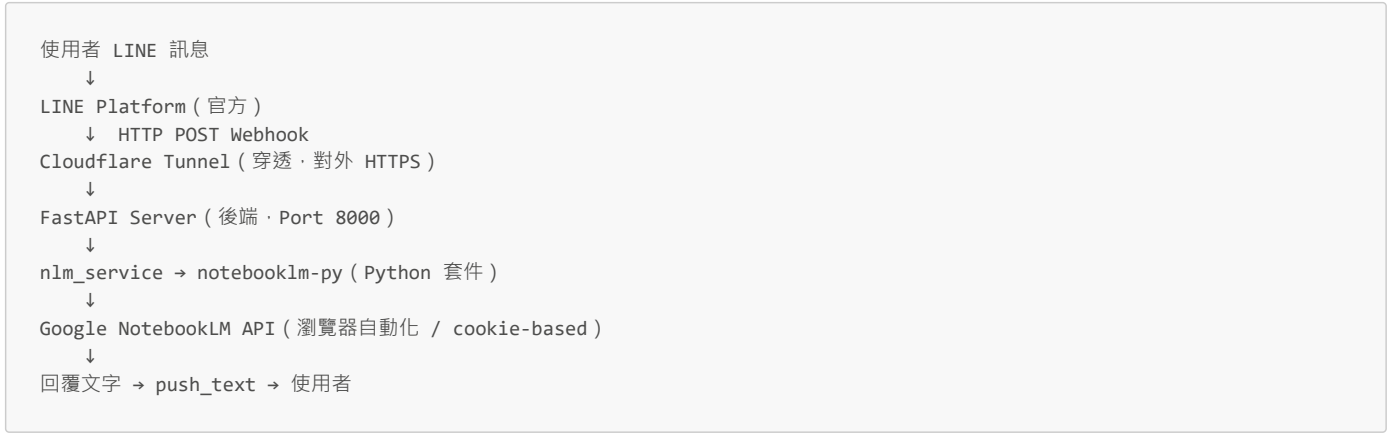
授權：GNU AGPL-3.0

專案概述

本專案是一套多租戶 LINE Bot 平台，讓每位課程學員各自持有一個 LINE 官方帳號，並綁定到自己的 Google NotebookLM Notebook，使用者只需透過 LINE 傳送訊息即可直接查詢 NotebookLM 的內容回覆。

系統架構

整體資料流



目錄結構

```
notebooklm-line/
├── backend/
│   ├── main.py           # FastAPI 應用程式進入點、lifespan 管理
│   ├── config.py        # 環境變數設定 (Pydantic Settings)
│   ├── database.py       # SQLite 初始化與 Schema Migration
│   ├── models.py         # Pydantic 資料模型
│   ├── requirements.txt  # Python 依賴套件
│   ├── requirements-test.txt # 測試依賴
│   ├── Dockerfile        # 後端容器映像定義
│   ├── .env.example      # 環境變數範例
│   └── routers/
│       ├── admin.py      # 管理 API (Channel CRUD、邀請碼、Notebook 管理)
│       ├── auth.py       # 認證 API (邀請碼驗證、NLM 登入綁定)
│       └── webhook.py     # LINE Webhook 接收與處理
│   └── services/
│       ├── nlm_service.py # NotebookLM 互動服務 (問答、列舉、綁定)
│       ├── line_service.py # LINE Messaging API (reply、push、loading)
│       ├── crypto_service.py # Fernet 對稱加密 (保護 cookie)
│       └── text_formatter.py # 回覆文字格式化 (分段、來源引用)
├── frontend/
│   └── (React 管理介面, build 後靜態檔由 FastAPI serve)
├── docker-compose.yml    # 容器編排 (backend + frontend)
├── .gitignore
└── LICENSE
```

核心模組說明

main.py — 應用程式進入點

- 使用 FastAPI `lifespan` context manager 在啟動時初始化 SQLite DB，並啟動背景定時任務。

- `expiry_scheduler`：每 60 秒執行一次 `cleanup_expired()`，自動刪除 `expires_at` 已過期的 Channel 及對應邀請碼。
- 掛載三支 Router：`admin (/api)`、`auth (/api)`、`webhook`（無前綴）。
- 如果存在 `static/` 或 `../frontend/dist/` 目錄，會自動以 SPA 模式 serve React 前端。

database.py — 資料庫

使用 `aiosqlite`（非同步 SQLite），包含兩張資料表：

資料表	主要欄位	說明
<code>channels</code>	<code>channel_id</code> 、 <code>channel_secret</code> 、 <code>channel_access_token</code> 、 <code>nlm_auth_json_encrypted</code> 、 <code>notebook_id</code> 、 <code>expires_at</code>	每個 LINE 官方帳號對應一筆 Channel，儲存加密後的 NLM cookie
<code>invite_codes</code>	<code>code</code> 、 <code>student_name</code> 、 <code>used</code> 、 <code>channel_id</code>	邀請碼管理，用於學員自助建立 Channel

支援 在地 **Schema Migration**：啟動時自動偵測並 ALTER TABLE 補上新欄位，不需手動執行 migration script。

routers/webhook.py — LINE Webhook

1. 接收 `POST /webhook/{channel_id}` 請求。
2. 查詢 DB 取得該 Channel 的 `channel_secret` 與 `channel_access_token`。
3. 使用 HMAC-SHA256 驗證 LINE 的 `x-line-signature` 標頭，防止偽造請求。
4. 對每個 `message` 類型事件：
 - 先呼叫 `show_loading()` 顯示載入動畫（或傳送「查詢中」提示）。
 - 以 `asyncio.create_task()` 在背景執行 `_ask_and_push()`，避免阻塞 Webhook 回應（LINE 要求 5 秒內回覆）。
5. `_ask_and_push()` 向 NotebookLM 查詢後，以 `push_text()` 將結果 push 給使用者。

services/nlm_service.py — NotebookLM 服務

- 使用 `notebooklm-py` 套件，透過 `storage_state.json`（Playwright 格式）模擬 Google 登入。
- `_nlm_lock (asyncio.Lock)`：因為 NLM client 透過環境變數 `NOTEBOOKLM_AUTH_JSON` 傳遞認證，此全域鎖確保同時只有一個請求覆寫環境變數，防止競態條件（Race Condition）。
- `ask_question()`：讀取加密 cookie → 解密 → 呼叫 `client.chat.ask()` → 取得 `answer` 與 `references` → 格式化並回傳訊息陣列。

services/crypto_service.py — 加密服務

使用 `cryptography.fernet` 對稱加密，以 `.env` 中的 `ENCRYPTION_KEY` 加密 NLM 的 `storage_state`（含 Google session cookie），確保 DB 中不存放明文憑證。

services/text_formatter.py — 訊息格式化

負責將 NotebookLM 回傳的 Markdown 格式答案轉換為 LINE 可讀的純文字，並另外組合引用來源的訊息（如 `[1]` 文件標題）。

環境設定

必要環境變數（`backend/.env`）

變數	說明	範例
<code>ENCRYPTION_KEY</code>	Fernet 對稱加密金鑰（必填）	<code>xxxxxxxxx=</code>
<code>WEBHOOK_BASE_URL</code>	Cloudflare Tunnel 對外 URL（必填）	<code>https://abc.trycloudflare.com</code>
<code>APP_PORT</code>	Docker 對外 Port（選填，預設 8083）	<code>8083</code>

產生加密金鑰指令：

```
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

部署說明

方式一：Docker Compose (推薦)

```
# 1. 複製 repo
git clone https://github.com/YwY170/notebooklm-line.git
cd notebooklm-line

# 2. 建立 backend/.env ( 參考 backend/.env.example )
cp backend/.env.example backend/.env
# 填入 ENCRYPTION_KEY 與 WEBHOOK_BASE_URL

# 3. 啟動
docker compose up -d --build

# 4. 啟動 Cloudflare Tunnel ( 另一個 terminal )
cloudflared tunnel --url http://localhost:8083
# 將產生的 URL 更新至 backend/.env 的 WEBHOOK_BASE_URL
```

Docker 服務說明：

服務	Container 名稱	說明
backend	n1m-line-backend	FastAPI · Port 8000 (僅內部)
frontend	n1m-line-frontend	Nginx serve React build · 對外 Port APP_PORT (預設 8083)

資料持久化：SQLite 資料庫掛載於 `./data/` 目錄。

方式二：本機直接執行

```
# 後端
cd backend
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8000

# 前端 ( 另一個 terminal )
cd frontend
npm install
npm run dev # 開發模式 ( proxy 到 localhost:8000 )

# Cloudflare Tunnel ( 另一個 terminal )
cloudflared tunnel --url http://localhost:8000
```

學員設定流程

- 開啟管理介面 (`https://your-tunnel-url`)。
- 輸入由管理員產生的邀請碼。
- 填入 LINE 官方帳號的 Channel ID、Channel Secret、Channel Access Token。
- 將介面顯示的 Webhook URL (格式： `{WEBHOOK_BASE_URL}/webhook/{channel_id}`) 貼至 LINE Developers Console。
- 在伺服器執行 `notebooklm login`，取得 `storage_state.json` 的內容後貼回管理介面，完成 NotebookLM 綁定。

管理員操作

批量產生邀請碼

```
curl -X POST "http://localhost:8000/api/invite-codes/generate?count=40"
```

查看所有 API 端點

啟動後訪問 `http://localhost:8000/docs` (Swagger UI)。

API 端點速查

方法	路徑	說明
POST	/api/verify-invite	驗證邀請碼
POST	/api/channels	建立 Channel
GET	/api/channels/{id}	查詢 Channel 資訊
POST	/api/channels/{id}/nlm-login	綁定 NotebookLM
GET	/api/channels/{id}/nlm-status	查詢綁定狀態
GET	/api/channels/{id}/notebooks	列出可用 Notebooks
POST	/api/channels/{id}/select-notebook	切換 Notebook
POST	/api/invite-codes/generate	批量產生邀請碼
POST	/webhook/{channel_id}	LINE Webhook 接收端

常見問題排查

問題現象	可能原因	處理方式
Webhook 回傳 403	LINE Signature 驗證失敗	確認 Channel Secret 填寫正確
Webhook 回傳 404	Channel ID 不存在	確認 Webhook URL 與 DB 中的 channel_id 一致
回覆「NotebookLM 尚未綁定」	nlm_auth_json_encrypted 為空	重新執行 notebooklm login 並更新 storage_state
回覆「查詢失敗」	NLM cookie 過期	重新登入並更新 storage_state
Docker 容器無法啟動	.env 缺少必要變數	確認 ENCRYPTION_KEY 已設定
Cloudflare URL 每次重啟都改變	使用臨時 Tunnel	改用固定 Tunnel 名稱或付費方案

注意事項與已知限制

- NLM 全域鎖**：目前以單一 `asyncio.Lock` 序列化所有 NotebookLM 請求，在高並發情況下會形成瓶頸。若學員數量增加，可考慮改為 per-channel 的 Lock 或連線池。
- Cookie 有效期**：Google session cookie 不保證永久有效，到期後需要學員重新登入並更新 storage_state。
- Cloudflare Tunnel**：使用 `cloudflared tunnel --url` 的臨時 URL 每次重啟會變更，需同步更新 LINE Developers Console 的 Webhook URL 及 .env 的 WEBHOOK_BASE_URL。
- SQLite 適用規模**：目前使用 SQLite，適合小規模課程（數十人以內）。若需大規模部署，建議遷移至 PostgreSQL。
- CORS 設定**：目前設為 `allow_origins=["*"]`，正式上線前建議限縮為特定網域。

依賴套件

後端（Python）

套件	用途
fastapi	Web 框架
uvicorn	ASGI 伺服器
aiosqlite	非同步 SQLite
notebooklm (notebooklm-py)	NotebookLM 自動化客戶端
cryptography	Fernet 加密
httpx	非同步 HTTP 請求（LINE API）

前端（Node.js / React）

前端為 React SPA，build 後靜態檔由 FastAPI 或 Nginx serve，詳見 frontend/ 目錄。

貢獻與溝通準則

本專案歡迎任何形式的技術交流與改進建議。若您發現程式架構有需要改進之處，歡迎透過 Issue 或 Pull Request 提出公開、具體、建設性的反饋。

本專案明確拒絕任何以私下或公開方式貶低、嘲諷原創作者開發瑕疵的行為。

每一段程式碼都是作者投入心力的成果，技術本就是在迭代中進步的。請以尊重與善意的方式進行交流，共同維護良好的開源協作環境。

本專案支持性別友善，明確拒絕任何形式的性別攻擊與歧視。

無論性別認同或性別表達為何，每位貢獻者與使用者都應受到平等尊重。